

Sep 17 - Sequence Alignment Algorithms

1.1 Edit Graph & Alignments, 1.2 Scoring Alignments, Scoring Scheme

Scoring scheme

- Given by matrix
- Includes unit scoring scheme

	A	T	C	G
A	1	0	0	0
T	0	1	0	0
C	0	0	1	0
G	0	0	0	1

$$X = x_1, x_2, \dots, x_M$$

$$Y = y_1, y_2, \dots, y_N$$

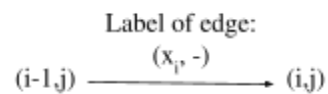
$$x_i, y_i \in \Sigma = \{A, C, G, T\}$$

Edit graph

- Given by $G_{X,Y}$
- Edge labelled directed graph (edges have directions)
- $G = (V, E)$ where V = vertex set and E = edge set
- **Vertices**
 - $(M+1) * (N+1)$
 - (i,j) represents row i , column j
 - Start numbering from 0
- **Edges**

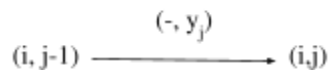
○ Gap in Y

- $1 \leq i \leq M$
- $1 \leq j \leq N$



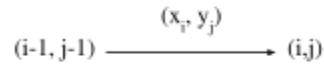
○ Gap in X

- $1 \leq i \leq N$
- $1 \leq j \leq M$



○ Alignment edge

- $1 \leq i \leq M$
- $1 \leq j \leq N$



- Conclusion
 - Only 3 types of edges coming to (i, j)
- Theorem
 - There is a one to one correspondence between directed paths from $(0, 0)$ to (M, N) and alignments between X and Y
 - Any alignment is a directed path
 - **DAG** - directed acyclic graph

Dynamic programming

- Def. Denote $S(i, j)$ the score of maximum path from begin $(0, 0)$ to (i, j)
- Because there are only 3 edges directed into vertex (i, j) it follows that any optimal max path P_{opt} to (i, j) must use one of these edges:
 - P_{opt} first arrives at $(i-1, j)$ and it follows a gap in Y edge into (i, j)
 - P_{opt} first arrives at $(i, j-1)$ and it follow a gap in X edge into (i, j)
 - P_{opt} first arrives at $(i-1, j-1)$ and it follows an alignment edge into (i, j)
- Conclusion

$$S(i, j) = \text{MAX} \begin{cases} S(i-1, j-1) + \delta(x_i, y_j), \\ S(i-1, j) + \delta(x_i, -), \\ S(i, j-1) + \delta(-, y_j), \end{cases}$$

- We need to compute $S(i, j)$ such that at any time we have already computed the values of S needed by the recurrence (at least one of the indices i and j smaller)
 - There are several ways to do such a computation $\rightarrow$ we will do it column by column



Global alignment algorithm

```

S[0,0] = 0
for j=1 to N:
    S[0,j] = S[0,j-1] + delta(-,y_j)
for i=1 to M:
    S[i,0] = S[i-1,0] + delta(x_i,-)
    for j=1 to N:
        S[i,j] = max(S[i-1,j-1] + delta(x_i,y_j), # alignment
                    S[i-1,j] + delta(x_i,-), # gap in Y
                    S[i,j-1] + delta(-,y_j)) # gap in X

print("Max Score", S[M,N])

```

Ungapped multiple alignment

B A B A

A A A C

A A C C

A A B A

A A C C

A A B C

- There are 24 amino acids observed

- 14 are A
- 4 are B
- 6 are C

- The observed proportions

- A: 14/24
- B: 4/24
- C: 6/24

- $4 \times (6 \text{ choose } 2) = 4 (6 \times 5 / 2) = 60$

Aligned pair	Frequency observed
A to A	26 / 60
A to B	8 / 60
A to C	10 / 60
B to B	3 / 60
B to C	6 / 60
C to C	7 / 60

- Compare observed frequencies to the expected frequencies
 - Need random model to compute expectations
- We will generate random sequences using the frequencies of aa (A 14/24, B, 4/24, ...)
 - That is, we choose at random sequences with same frequency of amino acids given above, and make an alignment of them
 - In a random alignment, the expected proportions of pairs is:
 - A to A $\Rightarrow (14/24) \times (14/24)$ (independence)
 - A to B $\Rightarrow 2 \times (14/24) \times (4/24)$
 - A to C ...
 - These fractions are used to calculate the “**estimated likelihood ratios**”
 - $2 \log_2 (\text{observed} / \text{expected})$

Aligned pair	Observed frequency	Expected frequency	$2 \log_2$ (Obs. frequency / expected frequency)
A to A	26 / 60	196 / 576	0.70
A to B	8 / 60	112 / 576	-1.09
A to C	10 / 60	168 / 576	-1.61
B to B	3 / 60	16 / 576	1.70
B to C	6 / 60	48 / 576	0.53
C to C	7 / 60	36 / 576	1.83

Scoring scheme:

	A	B	C
A	1	-1	-2
B	-1	2	1
C	-2	1	2